

PROGRAMACIÓN AVANZADA

Colecciones



Ing. Matias Macrino

COLECCIONES

- ▶ Una colección es un objeto que agrupa varios elementos en una sola unidad.
- ▶ Estas están definidas dentro de Collection Framework.



COLECCIONES

Collection Framework:

❖ Es una arquitectura definida para manejar y representar una colección de datos

❖ **Interfaces:**

- Tipo de datos abstracto para representar colecciones.
- Permite manipular una colección independiente de su implementación.

❖ **Implementaciones:**

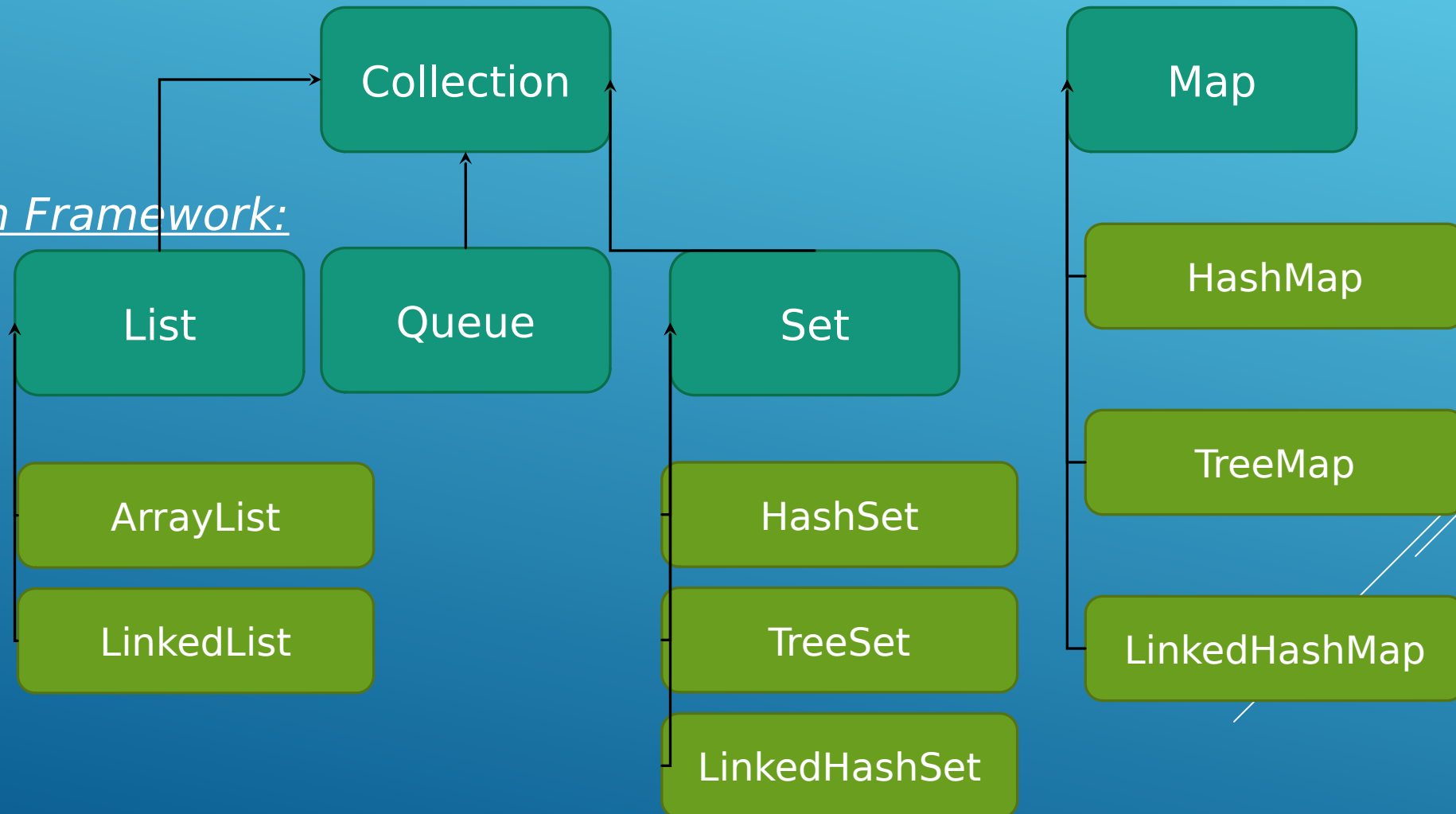
- ✓ Son implementaciones concretas de las colecciones de las interfaces.

❖ **Algoritmos:**

- ✓ Son métodos que permiten realizar cálculos útiles como realizar búsquedas u ordenar objetos.
- ✓ Son implementaciones polimórficas porque se pueden utilizar con distintas implementaciones.

COLECCIONES

Collection Framework:



COLECCIONES

Interface Set:

- ❖ Es una colecciones desordenada sin elementos duplicados.
- ❖ Incluye los mismos métodos que la interfaz collection.
 - ✓ El método agregar no permite elementos ya existentes.
- ❖ Se encuentra dentro de “java.util”. (Interface Set<E>, donde E es el tipo de elemento a contener).

COLECCIONES

Implementación Set:

❖ **HashSet:**

- ✓ Es una tabla de Hash con elementos desordenados.

❖ **TreeSet:**

- ✓ Es una estructura de árbol con elementos ordenados.
- ✓ Es mas lenta que HashSet.

❖ **LinkedHashSet:**

- ❖ Es una tabla de Hash con una lista enlazada, con elementos ordenados según el ingreso.

COLECCIONES

Interface List:

- ❖ Colección ordenada que permite elementos duplicados.
- ❖ Se encuentra dentro de “java.util”. Interface List<E>, donde E es el tipo de elemento a contener.

COLECCIONES

Implementación List:

❖ **ArrayList:**

- ✓ Es un arreglo redimensionable a diferencia de un arreglo convencional.
- ✓ Objetos no sincronizables.

❖ **LinkedList:**

- ✓ Es una lista enlazada de objetos.
- ✓ Puede utilizarse para implementar colas, pilas, arboles y listas doblemente enlazadas.
- ✓ Cada elemento se agrega al final de la lista.

COLECCIONES

Interface Map:

- ❖ Mapea claves asociados a un valor (Ejemplo: (1 clave, 1 valor)).
- ❖ No pueden contener elementos duplicados.
- ❖ Se encuentra dentro de “java.util”. Interface Map<K,V>, donde K define el tipo de la clave que mantendrá el Map y V es el valor del tipo que contendrá el Map.

COLECCIONES

Implementación Map:

❖ **HashMap:**

- ✓ Guarda elementos desordenados.

❖ **TreeMap:**

- ✓ Itera los elementos según el árbol según la implementación del método “compareTo()” e implementa la interfaz SortedMap para el método de ordenamiento.

❖ **LinkedHashMap:**

- ✓ Itera los elementos según el orden en el que fueron agregados a la estructura.

COLECCIONES

ArrayList <T>:

Un ArrayList representa una colección basadas en índices, en la que cada objeto de la misma tiene asociado un número (índice) según la posición que ocupa dentro de la colección, siendo cero la posición del primer elemento.

<T> es utilizado para indicar el tipo de dato que contendrá la colección.

❖ Declaracion de un ArrayList:

- ✓ `ArrayList <T> nombre_variable = new ArrayList <T> ();`
- ✓ `ArrayList <String> nombres = new ArrayList <String> ();`
- ✓ `ArrayList <String> nombres = new ArrayList <String> (20);`

COLECCIONES

ArrayList <T> Métodos:

❖ **Añadir elementos:**

- ✓ Se utiliza el método “boolean add(Object obj)”. Se agrega el nuevo elemento al final de la colección.
- ✓ nombres.add(“Rosario”);

❖ **Modificar un elemento:**

- ✓ Para modificar elementos la clase ArrayList nos provee el siguiente método set(int i, Object o).
- ✓ nombres.set(0, “Pedro”);

❖ **Eliminar un elemento:**

- ✓ Se utiliza el siguiente método “Object remove(int i)”.
- ✓ nombres.remove(i);
- ✓ Si “i” es un índice válido, elimina el elemento de la colección. Desplaza a la izquierda los elementos siguientes. Además devuelve el objeto eliminado. En caso contrario, esto genera una excepción.

COLECCIONES

ArrayList <T> Metodos:

❖ **Obtener un elemento:**

- ✓ Se utiliza el método "Object get(int i)". Devuelve el objeto ubicado en la posición, hay que tener en cuenta que devuelve un tipo object, por lo tanto para guardar la referencia del objeto en una variable, se debe de realizar una conversión explícita.
- ✓ nombres.get(i);

❖ **Obtener el tamaño de la colección:**

- ✓ Se utiliza el método "int size()". Devuelve la cantidad de elementos que hay dentro de la colección.
- ✓ nombres.size();

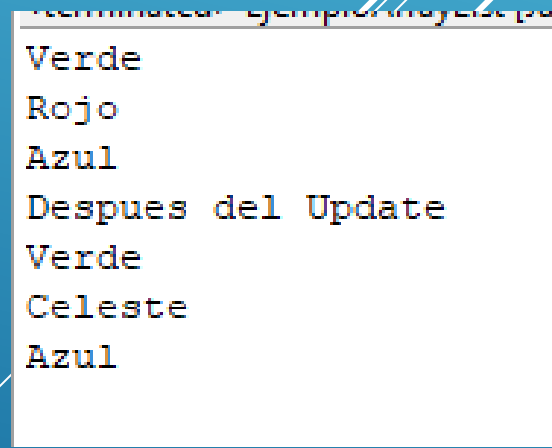
❖ **Otros métodos:**

- ❖ Copiar una colección: lista.clone();
- ❖ Eliminar todos los datos: lista.clear();
- ❖ Verificar si la lista esta vacía: lista.isEmpty();

COLECCIONES

```
public class EjemploArrayList {  
  
    public static void recorrerLista(ArrayList<String> lista) {  
        for(int i = 0; i < lista.size(); i++) {  
            System.out.println(lista.get(i).toString());  
        }  
    }  
  
    public static void main(String[] args) {  
        ArrayList<String> listaString = new ArrayList<String>();  
        listaString.add("Rojo");  
        listaString.add(0, "Verde");  
        listaString.add("Azul");  
  
        recorrerLista(listaString);  
        listaString.set(1, "Celeste");  
        System.out.println("Despues del Update");  
        recorrerLista(listaString);  
    }  
}
```

*Resultado por
consola:*



```
Verde  
Rojo  
Azul  
Despues del Update  
Verde  
Celeste  
Azul
```

COLECCIONES

```
public class EjemploArrayList {  
  
    public static void recorrerLista(List<String> lista){  
        for(String str : lista){  
            System.out.println(str);  
        }  
    }  
  
    public static void main(String[] args) {  
        List<String> listaString = new ArrayList<String>();  
        listaString.add("Rojo");  
        listaString.add(0, "Verde");  
        listaString.add("Azul");  
  
        recorrerLista(listaString);  
        listaString.set(1, "Celeste");  
        System.out.println("Despues del Update");  
        recorrerLista(listaString);  
    }  
}
```

 **Uso de Foreach**

Resultado por consola:

```
Verde  
Rojo  
Azul  
Despues del Update  
Verde  
Celeste  
Azul
```

COLECCIONES

HashTable:

Representa un tipo de colección basada en claves, donde los objetos almacenados en la misma (valores) no tienen asociados un índice numérico basado en su posición, sino una clave que lo identifica de forma única dentro de la colección.

❖ Declaración de un HashTable:

- ✓ Hashtable variable_objeto = new Hashtable();
- ✓ Hashtable tb = new Hashtable();

COLECCIONES

HashTable Métodos:

❖ **Añadir elementos:**

- ✓ Se utiliza el método “Object put(Object key, Object value)”. Agrega el objeto value, asignándole la clave especificada por Key.
- ✓ En el caso de existir la clave el nuevo value será sustituido por el nuevo elemento a agregar.
- ✓ `tb.put(“Key1”, “Objeto a agregar”);`

❖ **Obtener un elemento:**

- ✓ Para obtener un elemento de la colección se utiliza el método “Object get(Object Key)”.
- ✓ `tb.get(“Key1”);`

❖ **Eliminar un elemento:**

- ✓ Se utiliza el siguiente método “Object remove(Object Key)”.
- ✓ `tb.remove(“Key1”);`

❖ **Otros Métodos:**

- ✓ Obtener el numero de elementos “int size()”: `tb.size();`
- ✓ Existencia de elemento “boolean containsKey(Object Key)”: `tb.containsKey(“Key1”);`

COLECCIONES

HashTable recorrido:

❖ Recorrer la colección:

- ✓ Para esto se utiliza el siguiente método "Enumeration Keys()". El mismo devuelve un objeto enumeration

```
public void muestraDatos (Hashtable tb) {  
    String valor, clave;  
    Enumeration e = tb.keys();  
    while (e.hasMoreElements()) {  
        clave = (String)e.nextElement();  
        //Obtiene el objeto a partir de la clave  
        valor = (String)tb.get(clave);  
        System.out.println(valor);  
    }  
}
```

COLECCIONES

ArrayList y Hashtable Ejercitación:

- *Generar una clase, que se llamara “Auto” con los siguientes atributos marca, modelo, año, km y dominio.*
- *Estos mismos, serán almacenados en una implementación de ArrayList, generando los siguientes métodos:*
 - ✓ *Ver todos los autos de la colección.*
 - ✓ *Todos los autos de una marca determinada.*
 - ✓ *Todos los autos con menos de un determinado kilometraje.*
- *Implementar los mismos métodos utilizando la implementación de HashTable.*

PROGRAMACIÓN AVANZADA

Excepciones

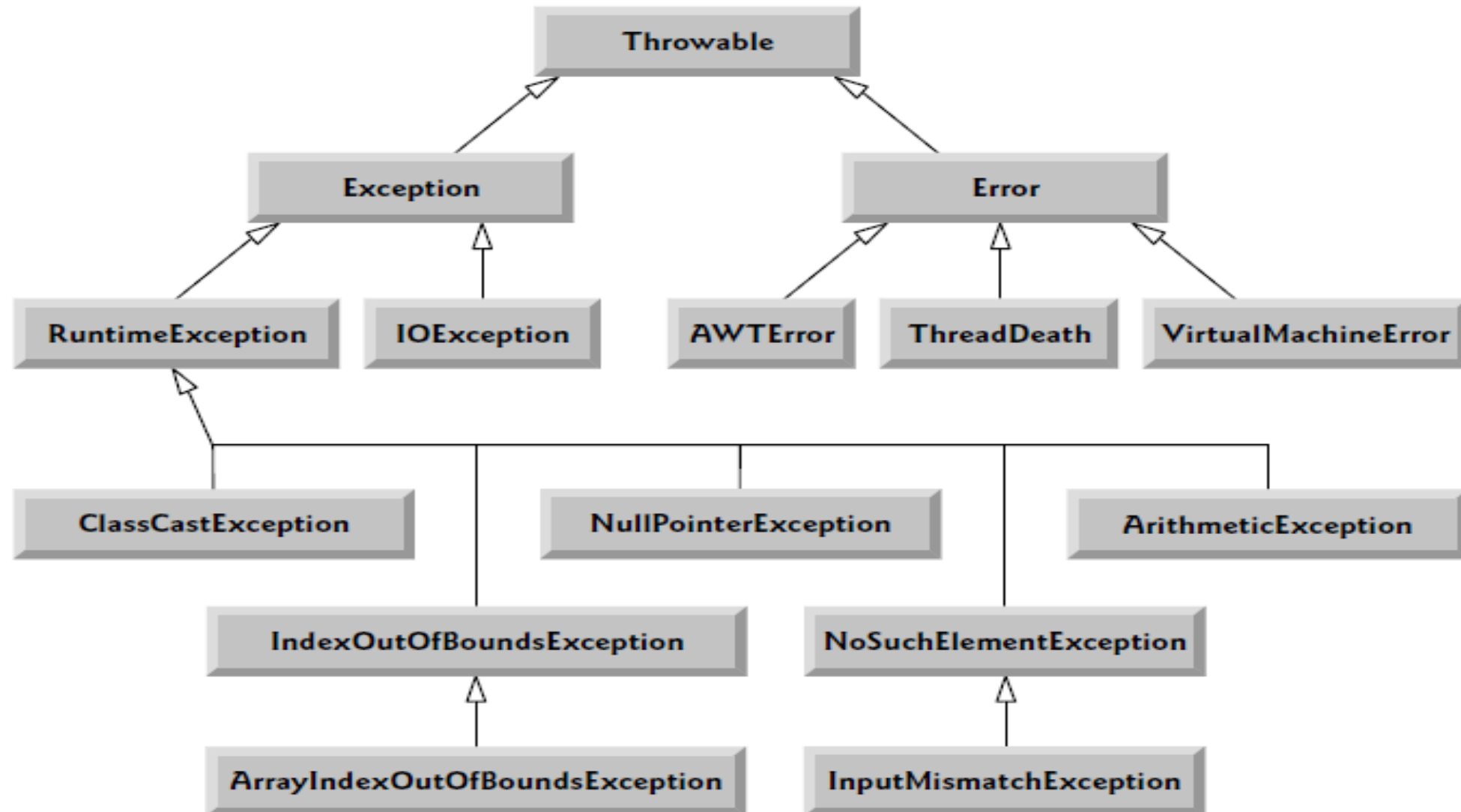
Several thin, parallel white lines are drawn diagonally across the bottom right portion of the slide, starting from the bottom edge and extending towards the top right corner.

EXCEPCIONES

► Excepciones y Errores:

- Una excepción es una situación anómala que puede producirse durante la ejecución de un programa.
Mediante la captura de excepciones, se proporciona un mecanismo que permite al programa sobreponerse a estas situaciones.
- Un error representa una situación anómala irreversible, como puede ser un fallo de JVM.
Por regla general, un programa no deberá intentar recuperarse de un error, dado que son situaciones que escapan al control del desarrollador.

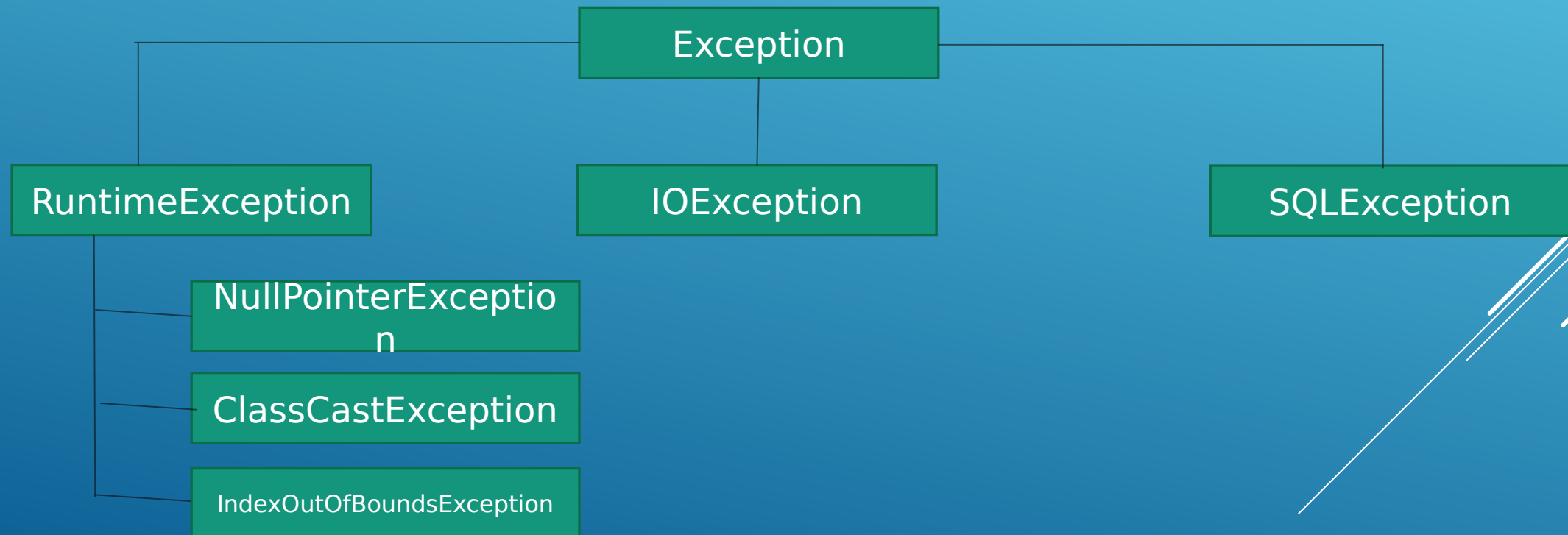
EXCEPCIONES



EXCEPCIONES

► Clases de Excepciones:

- Al producirse una excepción en un programa se crea un objeto de la subclase Exception a la que pertenece la excepción.



EXCEPCIONES

► Tipos de Excepciones:

Se dividen en dos grandes grupos:

- Excepciones Marcadas
- Excepciones no Marcadas

Excepciones Marcadas: son aquellas cuya captura es obligatoria. Este tipo de excepciones se producen al invocar a ciertos métodos de determinadas clases y son generadas (lanzadas) desde el interior de dichos métodos como consecuencia de algún fallo durante la ejecución de los mismos.

Un ejemplo es *IOException*. Esta es lanzada por el método “*readLine()*” de la clase “*BufferedReader*” cuando se produce un error durante la operación de lectura.

EXCEPCIONES

► Tipos de Excepciones:

Excepciones no Marcadas: son aquellas que se producen en tiempo de ejecución, es decir, "*RuntimeException*" y todas sus subclases.

No es obligatorio la captura de las mismas, el motivo es que gran parte de ellas (*NullPointerException*, *ClassCastException*, etc) se producen como consecuencia de un mal desarrollo.

Si durante la ejecución se produce una excepción y ésta no fue capturada, la JVM provoca la finalización inmediata, enviando a la consola el volcado de pila con los datos de la excepción.

EXCEPCIONES

► Declaración de una Excepciones:

Para la declaración de una excepción se utiliza la palabra “**throws**”, seguida de la lista de excepciones que el método puede provocar.

Ejemplo:

```
public String readLine() throws IOException { .... }
```

```
public void service () throws ServletException, IOException { .... }
```

EXCEPCIONES

► Captura de Excepciones:

► *Los bloques try...catch...finally:*

Las instrucciones **try**, **catch** y **finally** proporcionan una forma estructurada de captura excepciones dentro del programa evitando la utilización de instrucciones de control que dificultarían la lectura del código y lo harían mas propenso a errores.

► **Bloque try:**

- Delimita aquellas instrucciones donde se puede producir una excepción.
- Al momento de suceder una excepción, el control del programa se transfiere al bloque catch.

EXCEPCIONES

► Captura de Excepciones:

► **Bloque catch:**

- Define las instrucciones a ejecutar en caso de producirse una excepción.
- Se debe de tener en cuenta lo siguiente:
 - ✓ Se pueden definir tantos bloques catch como se considere necesario.
 - ✓ Aunque haya varios bloques, solo uno de ellos será ejecutado cuando se produzca la excepción.
 - ✓ Al existir varios bloques cuyas excepciones están relacionadas por la herencia, se debe de declarar de las excepciones mas especificas a las mas genéricas.
 - ✓ Estos bloques son opcionales, pero debe de existir un bloque **finally** en caso de no tener un bloque **finally** si o si se debe de tener un bloque catch.

EXCEPCIONES

► Captura de Excepciones:

► **Bloque catch:**

■ Ejemplos:

```
//Compilacion correcta
try {
    .
    .
    .
} catch (IOException e) {
    .
    .
    .
} catch (Exception e) {
    .
    .
    .
}

//No compila
try {
    .
    .
    .
} catch (Exception e) {
    .
    .
    .
} catch (IOException e) {
    .
    .
    .
}
```

EXCEPCIONES

► Captura de Excepciones:

► **Bloque finally:**

- Es opcional.
- Se ejecuta tanto como si se produce un excepción o no.
- Al producirse una excepción, se ejecuta este bloque después del bloque catch.
- Ejemplo:

```
try {  
    int a = 4/0;  
} catch(ArithmeticException e) {  
    System.out.println("división por 0");  
    return 0;  
} finally {  
    System.out.println("ejecutacion del finally");  
}
```

EXCEPCIONES

► Lanzamiento de una Excepción:

- Suele usarse para avisarle a otra parte del programa, que algo sucedió de forma anómala y no es posible continuar con la ejecución normal del método.
- Para hacer esto utilizamos la siguiente expresión:

❖ *throw objeto_excepción;*

□ Ejemplo:

```
try {  
    .  
    .  
    .  
} catch (Exception e) {  
    .  
    .  
    .  
    throw new Exception ("Ocurrio algo inesperado");  
}
```

EXCEPCIONES

► Ejemplo practico de Excepción:

```
public class DivisionEntreCero {  
  
    public static int cociente(int numerador, int denominador){  
        return numerador / denominador;  
    }  
  
    public static void main(String[] args) {  
        Scanner scan = new Scanner(System.in);  
        System.out.println("Ingresa numerador: ");  
        int numerador = scan.nextInt();  
        System.out.println("Ingresa denominador: ");  
        int denominador = scan.nextInt();  
  
        System.out.println("Resultado: " + cociente(numerador, denominador));  
    }  
}
```

EXCEPCIONES

▶ Métodos para el control de una Excepción:

- ▶ ***String getMessage():*** Devuelve un mensaje asociado a la excepción.
- ▶ ***void printStackTrace():*** Envía a la consola el volcado de la pila asociada a la excepción. Es muy útil durante el desarrollo.
- ▶ ***void printStackTrace(PrintStream s):*** Permite enviar el volcado de la pila a un objeto *PrintStream* cualquiera, por ejemplo, un fichero de log.

EXCEPCIONES

► Ejemplo practico de Excepción:

```
Ingrese numerador:  
10  
Ingrese denomiador:  
0  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at DivisionEntreCero.cociente(DivisionEntreCero.java:6)  
    at DivisionEntreCero.main(DivisionEntreCero.java:16)
```

Stack Trace (Rastreo de pila):

Es la información que se detalla en la consola en rojo, el cual es un mensaje descriptivo, que indica el problema que ocurrió y la pila de llamadas a métodos.

Incluye la ruta de ejecución que condujo a la excepción, método por método.

EXCEPCIONES

► Ejemplo practico de Excepción:

```
Ingrese numerador:  
10  
Ingrese denomiador:  
hola  
Exception in thread "main" java.util.InputMismatchException  
    at java.util.Scanner.throwFor(Unknown Source)  
    at java.util.Scanner.next(Unknown Source)  
    at java.util.Scanner.nextInt(Unknown Source)  
    at java.util.Scanner.nextInt(Unknown Source)  
    at DivisionEntreCero.main(DivisionEntreCero.java:14)
```

En los ejemplos podemos observar dos excepciones distintas una que es ***ArithmeticException*** y otra que es ***InputMismatchException***.

EXCEPCIONES

► Ejemplo practico de Excepción:

```
public class DivisionEntreCero {  
  
    public static int cociente(int numerador, int denominador){  
        return numerador / denominador;  
    }  
  
    public static void main(String[] args) {  
        Scanner scan = new Scanner(System.in);  
        System.out.println("Ingrese numerador: ");  
        int numerador = scan.nextInt();  
        System.out.println("Ingrese denomiador: ");  
        int denominador = scan.nextInt();  
  
        System.out.println("Resultado: " + cociente(numerador, denominador));  
    }  
}
```

Utilizar el manejo de excepciones para que no se nos den las dos excepciones vistas en los ejemplos.

EXCEPCIONES

► Clases de excepción personalizadas:

- Se utiliza para enviar una excepción que no exista dentro de las excepciones conocidas.
- Por ejemplo, en el desarrollo de una aplicación bancaria se puede crear una excepción personalizada “***saldoInsuficienteException***”, que será una subclase de ***Exception***.

```
public class SaldoInsuficienteException extends Exception {  
    public SaldoInsuficienteException(String message) {  
        super(message);  
    }  
}
```

EXCEPCIONES

► Clases de excepción personalizadas:

```
public class PruebaCustomException {  
    public static void main(String[] args) {  
        try {  
            throw new SaldoInsuficienteException("No posee saldo en la cuenta");  
        } catch (SaldoInsuficienteException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
SaldoInsuficienteException: No posee saldo en la cuenta  
    at PruebaCustomException.main(PruebaCustomException.java:8)  
|
```

EXCEPCIONES

► Clases de excepción personalizadas:

Una clase común de excepciones contiene solo cuatro constructores:

- Uno que no toma argumentos y pasa un objeto **String** como mensaje de error predeterminado al constructor de la superclase
- Uno que recibe un mensaje de error personalizado como un objeto String y lo pasa al constructor de la superclase
- Uno que recibe un mensaje de error personalizado como un objeto String y un objeto Throwable (para encadenar excepciones), y pasa ambos objetos al constructor de la superclase
- Uno que recibe un objeto Throwable (para encadenar excepciones) y pasa sólo este objeto al constructor de la superclase.

EXCEPCIONES

► Clases de excepción personalizadas:

► -

```
public class SaldoInsuficienteException extends Exception {  
  
    public SaldoInsuficienteException() {  
        super();  
    }  
  
    public SaldoInsuficienteException(String message, Throwable cause) {  
        super(message, cause);  
    }  
  
    public SaldoInsuficienteException(String message) {  
        super(message);  
    }  
  
    public SaldoInsuficienteException(Throwable cause) {  
        super(cause);  
    }  
  
}
```

EXCEPCIONES

▶ Try with resources:

- ▶ Esta mejora se introdujo a partir de la versión 7 de Java.
- ▶ El objetivo de esto fue que los recursos abiertos se cierren de manera automática y que no sea explícito por código a través de bloques **try - catch - finally**. Como también hacer más sencillo el código.
- ▶ Las clases que implementen la interfaz **AutoCloseable** pueden únicamente ser utilizadas dentro de este bloque **try with resources**.

EXCEPCIONES

► Try with resources:

```
public static String readFirstLineFromFileWithFinallyBlock(String path) throws IOException {  
    BufferedReader br = new BufferedReader(new FileReader(path));  
    try {  
        return br.readLine();  
    } finally {  
        if (br != null) br.close();  
    }  
}
```

Código en Java 1.6

```
public static String readFirstLineFromFile(String path) throws IOException {  
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {  
        return br.readLine();  
    }  
}
```

Código en Java 1.7 y superior.

PROGRAMACIÓN AVANZADA

API STREAM



Ing. Matias Macrino

API STREAM

▮ API Stream Java 8 – Definición:

- ▮ Es una **secuencia de elementos** de un **origen** que admite **operaciones concatenadas**.
- ▮ **Secuencia de elementos:** un Stream brinda una interfaz para un conjunto de valores de secuenciales de un tipo de elemento particular. No obstante, los Stream no almacenan elementos.
- ▮ **Origen:** los Stream toman su insumo de un origen de datos, tales como colecciones, matrices o recursos de E/S.
- ▮ **Operaciones concatenadas:** los Stream admiten operaciones al estilo SQL y operaciones comunes a la mayoría de los lenguajes de programación funcionales, como filtrar, buscar, coincidir, ordenar entre otras.

API STREAM

▮ API Stream Java 8:

- ▮ En toda aplicación Java desarrollada se crean y procesan colecciones. Las cuales son esenciales para muchas tareas a programar.
- ▮ Quien no se ha encontrado codificando operaciones de bucles una y otra vez, tan solo para obtener un único valor o el mayor valor de una colección.
- ▮ En el caso de usar colecciones enormes, para codificar operaciones de filtrado, conviene hacerlo a través del uso de multitarea (paralelizar). No obstante programar código en paralelo es una tarea ardua y en la que es fácil cometer errores.

API STREAM

- ▮ API Stream Java 8:
- ▮ En Java 8, se incorporo una interfaz en la API de Java, la cual es una nueva abstracción denominada **Stream**.
- ▮ Nos permite procesar los datos de un modo mas declarativo y eficiente, sin tantos bucles.
- ▮ Nos da la ventaja de aprovechar el procesamiento de multitarea, sin la necesidad de programar líneas de código enfocadas en el multiproceso.

API STREAM

▮ API Stream Java 8 - Ejemplo:

- ▮ Se necesita encontrar todas las operaciones del tipo GROCERY (Enumerador) y obtener

```
Transaccion tr = new Transaccion();
List<Transaccion> transacciones = tr.getAll();
List<Transaccion> groceryTransaction = new ArrayList<>();
//Filtro primero por tipo
for(Transaccion trx : transacciones) {
    if(trx.getType() == Transaccion.GROCERY) {
        groceryTransaction.add(trx);
    }
}
//Ordeno
Collections.sort(groceryTransaction, new Comparator<Transaccion>() {
    public int compare(Transaccion o1, Transaccion o2) {
        return o2.getValue().compareTo(o1.getValue());
    }
});

//Obtengo nueva lista
List<Integer> transaccionesID = new ArrayList<>();
for(Transaccion trxInt : groceryTransaction) {
    transaccionesID.add(trxInt.getId());
}
```

API STREAM

▮ API Stream Java 8 - Ejemplo:

▮ Utilizando la API de Stream:

```
List<Integer> transactionIDs = transacciones.stream().  
    filter(t -> t.getType() == Transaccion.GROCERY).  
    sorted(Comparator.comparing(Transaccion::getValue).reversed()).  
    map(Transaccion::getId).  
    collect(Collectors.toList());
```

API STREAM

▮ API Stream Java 8 - Ejemplo:

▮ Utilizando la API de Stream:

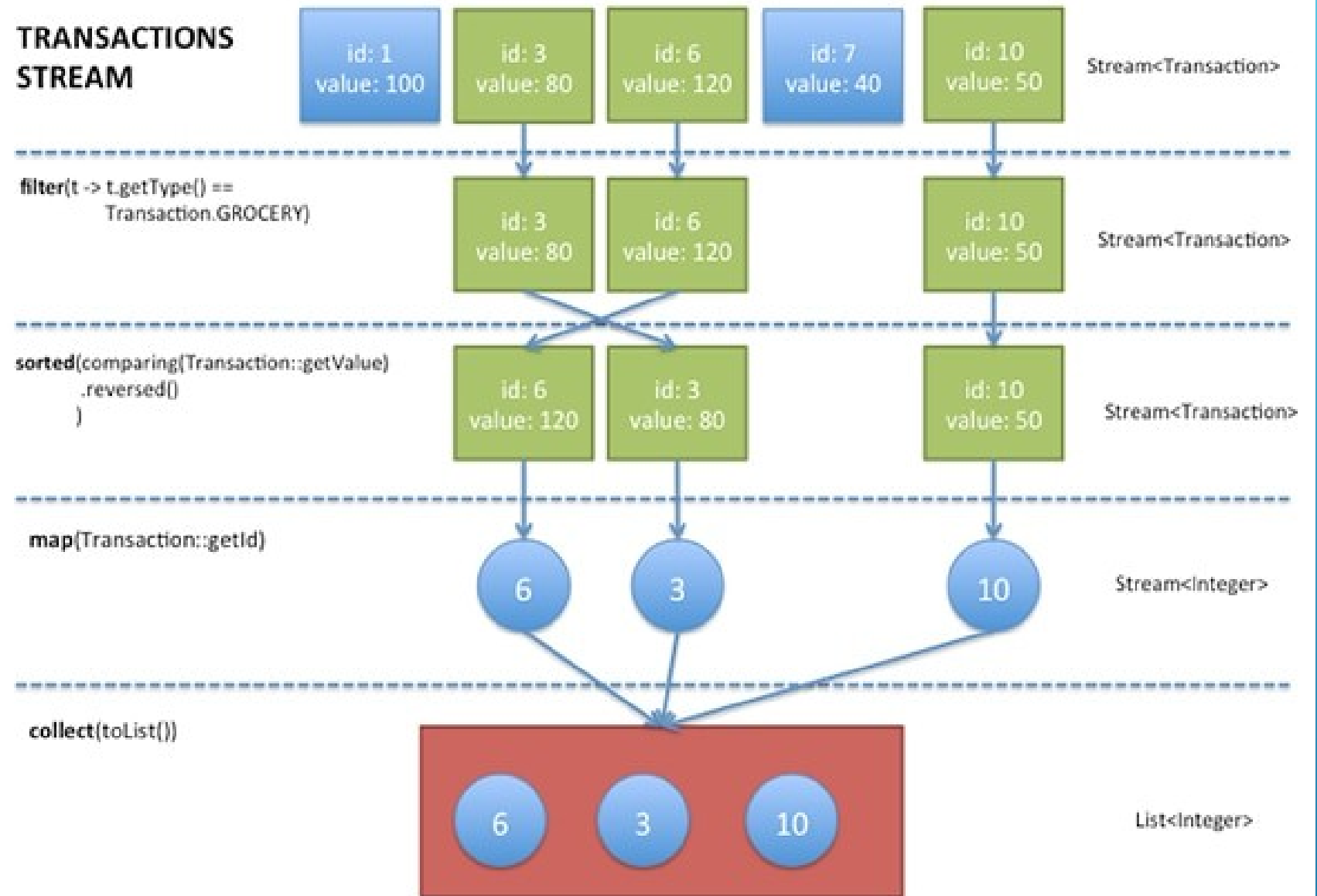
- ▮ En primer lugar obtendremos el **Stream** del listado a través del método **stream()**
- ▮ Luego se aplican una serie de operaciones concatenadas a nuestro **Stream**:
- ▮ **Filter:** filtra elementos según un predicado particular.
- ▮ **Sorted:** ordena los elementos según un comparador.
- ▮ **Map:** para extraer información.
- ▮ **Estas tres operaciones retornar un Stream**
- ▮ **Collect:** este comienza a abordar el proceso para devolver un resultado en forma de Colección. **No retorna un Stream, sino que en este caso es una lista.**

API STREAM

API Stream Java 8 - Ejemplo:

Utilizando la API de Stream:

**TRANSACTIONS
STREAM**



API STREAM

▮ Stream vs Colección:

▮ ¿Cuál es la diferencia?

- ▮ **Colecciones** hacen referencia a datos.
- ▮ **Stream** hacen referencia a cálculos.

▮ Ejemplo: una película almacenada en un DVD, se trata de una colección, porque contiene toda la estructura de datos. En cambio si el mismo video, lo reproducimos por internet, en esta caso hablamos de Stream, porque solo se necesita descargar solo unos bytes, así es posible mostrar los valores del comienzo del stream antes que la mayor parte del stream se haya computado.

▮ Es decir que la diferencia se relaciona cuando se hacen los cálculos.

API STREAM

▮ Stream Ejemplo practico:

- ▮ De una colección de enteros del 1 al 8 se desea obtener los números que den como resto de la división cero.

```
public static void main(String[] args) {  
    List<Integer> listaOriginal = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8);  
    List<Integer> listaNueva = listaOriginal.stream().  
        filter(n -> {  
            System.out.println("Filtro: " + n);  
            return n % 2 == 0;  
        }).  
        map(n -> {  
            System.out.println("Extraigo: " + n);  
            return n;  
        }).  
        collect(Collectors.toList());  
  
    System.out.println("Imprimo Lista final -----");  
    listaNueva.forEach(System.out::println);  
}
```

```
Filtro: 1  
Filtro: 2  
Extraigo: 2  
Filtro: 3  
Filtro: 4  
Extraigo: 4  
Filtro: 5  
Filtro: 6  
Extraigo: 6  
Filtro: 7  
Filtro: 8  
Extraigo: 8  
Imprimo Lista final ----  
2  
4  
6  
8
```

PROGRAMACIÓN AVANZADA

Expresiones Lambda



Ing. Matias Macrino

EXPRESIONES LAMBDA

▮ Expresiones Lambda:

- ▮ Junto con la API de **Stream** son una de las nuevas y mas importantes características de Java 8
- ▮ Se puede crear código mas conciso y significativo.
- ▮ Nos abre las puertas hacia la programación funcional en Java, en donde las funciones juegan un papel fundamental.
- ▮ Lambda junto a Stream son una combinación muy poderosa que requiere de un cambio de paradigma en la forma que hemos codificado en Java hasta el momento.

EXPRESIONES LAMBDA

▮ Expresiones Lambda:

- ▮ **Funciones como primer nivel:** uno de los conceptos de programación funcional habla de que las funciones (métodos) sean definidos como entidades de primer nivel, es decir que pueden aparecer en partes de nuestro código donde otras entidades de primer nivel (valores primitivos u objetos) aparecen.

▮ **Sintaxis básica: (parámetros) -> {cuerpo lambda}**

El operador **lambda (->)** separa la declaración de parámetros de la declaración del cuerpo de la función.

Parámetros: si es uno solo no es necesario utilizar paréntesis en caso contrario si.

Cuerpo de lambda: al igual que parámetros solo que con llaves y en caso de retornar valor se debe de incluir la palabra **return**

EXPRESIONES LAMBDA

- ▮ Expresiones Lambda:
 - ▮ **Tipos**, se pueden clasificar en:
 - ▮ **Consumidores**
 - ▮ **Proveedores**
 - ▮ **Funciones:**
 - ▮ **Operadores Unarios.**
 - ▮ **Operadores Binarios.**
 - ▮ **Predicados**

EXPRESIONES LAMBDA

▮ Expresiones Lambda – Tipos Consumidores:

- ▮ Se trata de aquellas expresiones que aceptan un solo valor y no retornan valor alguno.

- ▮ Ejemplo:

String mensaje -> System.Out.println(mensaje)

- ▮ Las expresiones ***BiConsumidores***, un caso excepcional, son aquellas que toman dos valores como parámetro y no retornan resultado.

- ▮ Ejemplo:

(String key, String value) -> System.Out.println("Key: %s, Value: %s %n", key, value)

EXPRESIONES LAMBDA

▮ Expresiones Lambda – Tipos

- ▮ **Proveedores:** se trata de expresiones que no tienen parámetros pero en este caso si retornan un resultado.

▮ Ejemplo:

() -> return createRandomInteger()

- ▮ **Predicados:** son aquellas expresiones que aceptan un parámetro y retornan un valor lógico

▮ Ejemplo:

String mensaje -> mensaje.length > 50

EXPRESIONES LAMBDA

▮ Expresiones Lambda – Tipos

- ▮ **Funciones:** aquellas expresiones que aceptan un argumento y retornan un valor como resultado y cuyos tipos no tienen porque ser iguales.
- ▮ Ejemplo:
Order persisteOrder -> persisteOrder.getIdentifier()
- ▮ **Operados Unarios:** caso especial en las que tanto el parámetro como el valor a retornar son del mismo tipo.
- ▮ Ejemplo: String mensaje -> mensaje.toLowerCase()
- ▮ **Operadores Binarios:** dos parámetros y el valor de retorno son del mismo tipo.
- ▮ Ejemplo: **(String mensaje, String mensaje2) -> mensaje.concat(mensaje2)**

EXPRESIONES LAMBDA

▮ Expresiones Lambda:

- ▮ **Referencia a métodos:** nos permite reutilizar un método como expresión lambda. Para hacer uso de las referencia a métodos basta con utilizar la siguiente sintaxis:

- ▮ *referenciaObjetivo::nombreDelMetodo*

- ▮ *Existen tres tipos:*

- ▮ *Métodos estáticos*
 - ▮ *Métodos de instancia de un tipo*
 - ▮ *Métodos de instancia de un objeto existente.*

EXPRESIONES LAMBDA

▮ Expresiones Lambda – Referencia a Metodos:

▮ Ejemplo Método estático:

- ▮ ***(String info) -> System.Out.println(info) //E.Lambda sin referencia.***
- ▮ ***System.Out::println //E.Lambda con referencia a un método estático.***

▮ Ejemplo Método de un tipo:

- ▮ ***(Student s, int registryIndex) -> student.getRegistry(registryIndex) //E.Lambda sin referencia***
- ▮ ***Student::getRegistry //E.Lambda con referencia a método de un tipo.***

▮ Ejemplo Método de un objeto existente:

- ▮ ***Student s -> getMarks(s) //E.Lambda sin referencia.***
- ▮ ***This::getMarks //E.Lambda con referencia a un método de un objeto existente.***